

COMPTE RENDU

Administration Réseaux et Programmation Système - TP5 - Tubes et
Signaux

3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignants : M. Noufel & M. ElAmrani
Date : 15 mars 2026

Table des matières

1	Exercice 1 : Tubes	2
1.1	Partie a — Fonction <code>nbOccu</code>	2
1.2	Partie b — Saisie utilisateur	2
1.3	Partie c — Communication par <code>exit()</code>	2
1.4	Partie d — Communication par tube	4
2	Exercice 2 : Signaux	6
2.1	Partie a — Dérouter <code>SIGTERM</code> et <code>SIGKILL</code>	6
2.2	Partie b — Distributeur de café	7
2.3	Partie c — Ping-pong de signaux	8
3	Exercice 3 : Synchronisation par signaux	10
	Conclusion	12

1 Exercice 1 : Tubes

L'objectif est de compter le nombre d'occurrences d'un caractère dans une chaîne de caractères en utilisant trois processus qui se partagent le parcours du tableau. Le tableau est déclaré en variable globale et chaque processus traite un tiers de la chaîne.

1.1 Partie a — Fonction nbOccu

La fonction `nbOccu` reçoit le caractère recherché, les indices de début et de fin, ainsi que la chaîne en paramètre. Elle parcourt la chaîne entre les indices `debut` (inclus) et `fin` (exclus) et incrémente un compteur à chaque occurrence trouvée.

Listing 1 – Fonction nbOccu

```
1 int nbOccu(char val, int debut, int fin, char chaine[]) {
2     int nb_occ = 0;
3     for (int i = debut ; i < fin ; i++) {
4         if (chaine[i] == val) {
5             nb_occ++;
6         }
7     }
8     return nb_occ;
9 }
```

Le tableau est passé en paramètre plutôt qu'accédé directement en global, ce qui rend la fonction plus générique et réutilisable sur n'importe quel tableau.

1.2 Partie b — Saisie utilisateur

Le `main` commence par demander à l'utilisateur la chaîne à analyser ainsi que le caractère à rechercher. On utilise `fgets` pour la chaîne (afin de gérer les espaces) et `scanf` avec l'espace avant `%c` pour absorber le retour à la ligne résiduel dans le buffer.

Listing 2 – Saisie utilisateur dans le main

```
1 printf("Entrez une chaine : ");
2 fgets(chaine, TAILLE_MAX, stdin);
3
4 printf("Entrez le caractere a rechercher : ");
5 scanf(" %c", &val);
```

1.3 Partie c — Communication par `exit()`

Le père crée deux fils avec `fork()`. Chaque processus traite un tiers de la chaîne : le fils 1 le tiers du milieu, le fils 2 le dernier tiers, et le père le premier tiers. Les fils transmettent leur résultat au père via `exit()`, que le père récupère avec `WEXITSTATUS()`.

Listing 3 – `ex1c.c` — Communication par `exit()`

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <unistd.h>
```

```

5 #include <sys/wait.h>
6
7 #define TAILLE_MAX 256
8 char chaine[TAILLE_MAX];
9
10 int nbOccu(char val, int debut, int fin, char chaine[]) {
11     int nb_occ = 0;
12     for (int i = debut ; i < fin ; i++) {
13         if (chaine[i] == val) {
14             nb_occ++;
15         }
16     }
17     return nb_occ;
18 }
19
20 int main(int argc, char* argv[]) {
21     char val;
22
23     printf("Entrez une chaine : ");
24     fgets(chaine, TAILLE_MAX, stdin);
25
26     printf("Entrez le caractere a rechercher : ");
27     scanf(" %c", &val);
28
29     int taille = strlen(chaine);
30     int tiers = taille / 3;
31
32     pid_t fils1, fils2;
33     int status1, status2;
34
35     fils1 = fork();
36     if (fils1 == 0) {
37         exit(nbOccu(val, tiers, 2*tiers, chaine));
38     }
39
40     fils2 = fork();
41     if (fils2 == 0) {
42         exit(nbOccu(val, 2*tiers, taille, chaine));
43     }
44
45     int res_pere = nbOccu(val, 0, tiers, chaine);
46
47     waitpid(fils1, &status1, 0);
48     waitpid(fils2, &status2, 0);
49
50     int res1 = WEXITSTATUS(status1);
51     int res2 = WEXITSTATUS(status2);
52
53     printf("Nombre d'occurrences de '%c' : %d\n", val, res_pere +
54           res1 + res2);
55     return 0;

```

Résultat d'exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP5/ex1$ ./ex1c
Entrez une chaîne : hello, i'm under the water, please help me
Entrez le caractère à rechercher : e
Nombre d'occurrences de 'e' : 8
```

FIGURE 1 – Exécution de ex1c : comptage par `exit()` et `WEXITSTATUS()`

Analyse

Le père calcule son propre tiers en parallèle pendant que les fils calculent les leurs, puis attend leurs résultats avec `waitpid()`. La valeur retournée par `exit()` est limitée à l'intervalle 0–255, ce qui est suffisant pour ce TP mais constitue une limitation par rapport à la version avec `tube`.

1.4 Partie d — Communication par tube

Dans cette version, les fils écrivent leur résultat dans un tube partagé via `write()`, et le père lit les deux résultats depuis l'autre extrémité avec `read()`. Le tube est créé avant le `fork()` pour être partagé entre les processus.

Listing 4 – ex1d.c — Communication par tube

```
1 int main(int argc, char* argv[]) {
2     char val;
3
4     printf("Entrez une chaîne : ");
5     fgets(chaine, TAILLE_MAX, stdin);
6
7     printf("Entrez le caractere a rechercher : ");
8     scanf(" %c", &val);
9
10    int taille = strlen(chaine);
11    int tiers = taille / 3;
12
13    int fd[2];
14    pipe(fd); /* fd[0] = lecture ; fd[1] = ecriture */
15
16    pid_t fils1, fils2;
17
18    fils1 = fork();
19    if (fils1 == 0) {
20        close(fd[0]);
21        int res = nbOccu(val, tiers, 2*tiers, chaine);
22        write(fd[1], &res, sizeof(int));
23        close(fd[1]);
24        exit(0);
```

```

25     }
26
27     fils2 = fork();
28     if (fils2 == 0) {
29         close(fd[0]);
30         int res = nbOccu(val, 2*tiers, taille, chaine);
31         write(fd[1], &res, sizeof(int));
32         close(fd[1]);
33         exit(0);
34     }
35
36     close(fd[1]);
37
38     int res_pere = nbOccu(val, 0, tiers, chaine);
39
40     int res1, res2;
41     read(fd[0], &res1, sizeof(int));
42     read(fd[0], &res2, sizeof(int));
43     close(fd[0]);
44
45     wait(NULL);
46     wait(NULL);
47
48     printf("Nombre d'occurrences de '%c' : %d\n", val, res_pere +
49           res1 + res2);
50     return 0;
51 }

```

Résultat d'exécution

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP5/ex1$ ./ex1d
Entrez une chaîne : hello, i'm under the water, please help me
Entrez le caractère à rechercher : e
Nombre d'occurrences de 'e' : 8

```

FIGURE 2 – Exécution de ex1d : comptage par tube, même résultat qu'avec exit()

Analyse

Le tube est un mécanisme IPC (Inter-Process Communication) unidirectionnel : ce qui est écrit dans `fd[1]` est lisible depuis `fd[0]`. Chaque processus ferme le bout du tube dont il n'a pas besoin — les fils ferment `fd[0]` et le père ferme `fd[1]` — sans quoi le `read()` du père ne se terminerait jamais, le tube ne serait jamais considéré comme fermé par le noyau. Contrairement à la version avec `exit()`, il n'y a ici aucune limitation sur la valeur transmise puisqu'on écrit un `int` entier.

	Version exit()	Version tube
Mécanisme	Code de retour du processus	IPC via descripteurs de fichiers
Limite de valeur	0–255	Aucune (int entier)
Flexibilité	Faible	Élevée

2 Exercice 2 : Signaux

2.1 Partie a — Dérouter SIGTERM et SIGKILL

Le fils installe un gestionnaire de signal (*handler*) pour SIGTERM et tente d'en installer un pour SIGKILL, puis se met en attente avec `pause()`. Le père lui envoie successivement SIGTERM puis SIGKILL.

Listing 5 – ex2a.c — Dérouter SIGTERM et SIGKILL

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5 #include <sys/wait.h>
6
7 void handler(int sig) {
8     printf("Fils (PID=%d) : signal %d reçu mais ignore\n", getpid
9         (), sig);
10 }
11
12 int main() {
13     pid_t fils = fork();
14
15     if (fils == 0) {
16         signal(SIGTERM, handler);
17         signal(SIGKILL, handler); /* sans effet, SIGKILL ne peut
18             pas être déroute */
19
20         printf("Fils (PID=%d) : en attente de signaux...\n",
21             getpid());
22         while(1) {
23             pause();
24         }
25     }
26
27     sleep(1);
28
29     printf("Père : envoi de SIGTERM au fils\n");
30     kill(fils, SIGTERM);
31     sleep(1);
32
33     printf("Père : envoi de SIGKILL au fils\n");
34     kill(fils, SIGKILL);
35
36     wait(NULL);
37     printf("Père : fils termine\n");
38
39     return 0;
40 }
```

Résultat d'exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP5/ex2$ ./ex2a
Fils (PID=131447) : en attente de signaux...
Père : envoi de SIGTERM au fils
Fils (PID=131447) : signal 15 reçu mais ignoré
Père : envoi de SIGKILL au fils
Père : fils terminé
```

FIGURE 3 – SIGTERM intercepté par le handler, SIGKILL tue le fils malgré tout

Analyse

SIGTERM (signal 15) est intercepté par le handler : le fils affiche le message et continue de s'exécuter. SIGKILL (signal 9) ne peut jamais être dérouté ni ignoré — c'est une règle absolue du noyau Linux. Le fils est tué immédiatement sans pouvoir exécuter aucun code. Cette distinction est fondamentale en pratique : SIGTERM permet à un processus de se terminer proprement (fermer des fichiers, libérer des ressources), tandis que SIGKILL est réservé aux cas où le processus ne répond plus.

2.2 Partie b — Distributeur de café

On simule un distributeur électronique de café avec deux processus : le père joue le rôle du serveur et le fils celui du client. La synchronisation est assurée uniquement par SIGSTOP et SIGCONT. Le client envoie SIGCONT au serveur pour signaler sa demande, puis se met en pause avec SIGSTOP. Le serveur, réveillé, sert le verre puis envoie SIGCONT au client et se remet en pause.

Listing 6 – ex2b.c — Distributeur de café avec SIGSTOP/SIGCONT

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5 #include <sys/wait.h>
6
7 int main() {
8     pid_t pid_fils;
9     pid_t pid_pere = getpid();
10
11     pid_fils = fork();
12
13     if (pid_fils == 0) {
14         /* === Client === */
15         pid_pere = getppid();
16
17         printf("Client : demanderverre(1)\n");
18         kill(pid_pere, SIGCONT);      /* reveille le pere */
19         kill(getpid(), SIGSTOP);      /* le fils se met en pause */
20
21         printf("Client : demanderverre(2)\n");
```



```

22     kill(pid_pere, SIGCONT);      /* reveille le pere */
23     kill(getpid(), SIGSTOP);     /* le fils se met en pause */
24
25     printf("Client : j'ai eu mes deux verres, merci !\n");
26     exit(0);
27 }
28
29 /* === Serveur === */
30 kill(getpid(), SIGSTOP);        /* se met en pause, attend la
    demande */
31
32 printf("Serveur : servirverre(1)\n");
33 kill(pid_fils, SIGCONT);        /* reveille le fils */
34 kill(getpid(), SIGSTOP);        /* le pere se met en pause */
35
36 printf("Serveur : servirverre(2)\n");
37 kill(pid_fils, SIGCONT);        /* reveille le fils */
38
39 wait(NULL);
40 return 0;
41 }

```

Résultat d'exécution

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP5/ex2$ ./ex2b
Client : demanderverre(1)
Serveur : servirverre(1)
Client : demanderverre(2)
Serveur : servirverre(2)
Client : j'ai eu mes deux verres, merci !
[1]+  Arrêté                  ./ex2b
[1]+  Fini                    ./ex2b

```

FIGURE 4 – Synchronisation correcte : demanderverre(1) → servirverre(1) → demanderverre(2) → servirverre(2)

Analyse

La synchronisation repose sur un mécanisme d'alternance : chaque processus se met en pause avec SIGSTOP après avoir effectué sa tâche, et réveille l'autre avec SIGCONT. Cela garantit l'ordre strict demandé par l'énoncé sans avoir recours à un signal de notification supplémentaire. On peut modéliser ce comportement comme un automate à états finis où les transitions sont déclenchées par les signaux.

2.3 Partie c — Ping-pong de signaux

Le père envoie SIGUSR2 au fils, le fils répond avec SIGUSR1 au père, et ainsi de suite. Chaque processus installe un handler qui, à la réception d'un signal, renvoie le

signal correspondant à l'autre processus. Le père envoie le premier signal pour démarrer l'échange.

Listing 7 – ex2c.c — Ping-pong SIGUSR1/SIGUSR2

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5 #include <sys/wait.h>
6
7 pid_t pid_pere, pid_fils;
8
9 void handler_pere(int sig) {
10     printf("Pere : reçu SIGUSR1, renvoie SIGUSR2\n");
11     sleep(1);
12     kill(pid_fils, SIGUSR2);
13 }
14
15 void handler_fils(int sig) {
16     printf("Fils : reçu SIGUSR2, renvoie SIGUSR1\n");
17     sleep(1);
18     kill(pid_pere, SIGUSR1);
19 }
20
21 int main() {
22     pid_pere = getpid();
23
24     pid_fils = fork();
25
26     if (pid_fils == 0) {
27         /* === FILS === */
28         pid_pere = getppid();
29         signal(SIGUSR2, handler_fils);
30         pause(); /* attend le premier signal du pere */
31         exit(0);
32     }
33
34     /* === PERE === */
35     signal(SIGUSR1, handler_pere);
36
37     printf("Pere : envoi du premier SIGUSR2\n");
38     sleep(1);
39     kill(pid_fils, SIGUSR2); /* pere démarre le ping-pong */
40
41     pause(); /* attend SIGUSR1 du fils */
42
43     wait(NULL);
44     return 0;
45 }
```

Résultat d'exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP5/ex2$ ./ex2c
Père : envoi du premier SIGUSR2
Fils : reçu SIGUSR2, renvoie SIGUSR1
Père : reçu SIGUSR1, renvoie SIGUSR2
```

FIGURE 5 – Un échange complet : père envoie SIGUSR2, fils répond SIGUSR1, père renvoie SIGUSR2

Analyse

Le handler est indispensable ici car le comportement par défaut de SIGUSR1 et SIGUSR2 est de terminer le processus. Sans handler, le premier signal tuerait le destinataire. Les deux handlers se rappellent mutuellement, créant un échange infini tant que les deux processus restent vivants. Le `pause()` est nécessaire pour que le processus ne se termine pas avant de recevoir le premier signal.

3 Exercice 3 : Synchronisation par signaux

L'objectif est de mettre en œuvre une chaîne de synchronisation entre trois processus. P1 crée P2, P2 crée P3. P3 signale sa création à P1 via SIGUSR1 puis se termine. P2 détecte la mort de P3 avec `wait()`, signale cet événement à P1 via SIGUSR2 puis se termine. P1 doit traiter les événements dans l'ordre : *P3 créé* → *P3 terminé* → *P2 terminé*.

Listing 8 – ex3.c — Synchronisation P1/P2/P3 par signaux

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5 #include <sys/wait.h>
6
7 pid_t pid_P1;
8
9 void handler_P1(int sig) {
10     if (sig == SIGUSR1) {
11         printf("Processus P3 cree\n");
12     } else if (sig == SIGUSR2) {
13         printf("Processus P3 termine\n");
14     }
15 }
16
17 int main() {
18     pid_P1 = getpid();
19
20     signal(SIGUSR1, handler_P1);
21     signal(SIGUSR2, handler_P1);
22
23     pid_t pid_P2 = fork(); /* P1 cree P2 */
```

```

24
25     if (pid_P2 == 0) {
26         /* === P2 === */
27         pid_t pid_P3 = fork(); /* P2 cree P3 */
28
29         if (pid_P3 == 0) {
30             /* === P3 === */
31             kill(pid_P1, SIGUSR1); /* signale sa creation a P1 */
32             exit(0);               /* se termine */
33         }
34
35         /* P2 attend la mort de P3 */
36         wait(NULL);
37
38         /* P2 signale la mort de P3 a P1 */
39         kill(pid_P1, SIGUSR2);
40         exit(0);
41     }
42
43     /* === P1 === */
44     wait(NULL); /* attend la mort de P2 */
45     printf("Processus P2 termine\n");
46
47     return 0;
48 }

```

Résultat d'exécution

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP5/ex3$ ./ex3
Processus P3 cree
Processus P3 termine
Processus P2 termine

```

FIGURE 6 – Ordre correct : P3 créé → P3 terminé → P2 terminé

Analyse

Plusieurs points sont essentiels dans cette implémentation. Le PID de P1 est stocké dans une variable globale `pid_P1` avant le `fork()` pour que P2 et P3 y aient accès par héritage. Les handlers sont également installés avant le `fork()` pour éviter une fenêtre de vulnérabilité entre la création de P3 et l'installation du handler : si P3 envoyait SIGUSR1 avant que P1 n'ait installé son handler, le comportement par défaut (terminer le processus) s'appliquerait. Enfin, P2 utilise `wait()` pour détecter la mort de P3 avant d'envoyer SIGUSR2 à P1, garantissant ainsi l'ordre strict des événements.

Conclusion

Ce TP5 a permis d'explorer deux mécanismes fondamentaux de communication inter-processus en programmation système. L'exercice 1 a illustré la différence entre la communication par code de retour (`exit/WEXITSTATUS`), limitée à des valeurs entre 0 et 255, et la communication par tube, plus flexible et adaptée à des données de taille arbitraire. Les exercices 2 et 3 ont introduit les signaux comme mécanisme de synchronisation et de notification entre processus, avec une distinction importante entre les signaux interceptables (`SIGTERM`, `SIGUSR1`, `SIGUSR2`) et le signal `SIGKILL` qui ne peut jamais être ignoré ou redirigé. La combinaison de `wait()`, `kill()` et des handlers de signaux permet de construire des synchronisations précises entre processus, comme illustré par le distributeur de café et la chaîne P1/P2/P3.